

ADC and Analog Interfacing

Introduction to ARM Cortex-M Microprocessors — Chapter 11

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

4 Design Challenge

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

4 Design Challenge

Temperature, light, sound, force, and position are all continuous quantities. A processor can only work with numbers. The analog-to-digital converter is the instrument that crosses that gap. Using it well is an engineering problem, not just a programming one. You must decide how fast to sample, how much resolution you need, and how much of the answer you can actually trust.

After completing this chapter, you should be able to:

1. Explain sampling, quantization, and resolution, and apply the Nyquist criterion to choose a sampling rate.
2. Calculate the quantization step size and convert between digital codes and physical voltages.
3. Configure a GPIO pin for analog input and initialize the ADC peripheral.
4. Acquire a reading by polling and by interrupt, and justify the choice between them.
5. Select an appropriate sampling time for a given source impedance.
6. Scale a raw ADC reading into engineering units for a real sensor.

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

4 Design Challenge

Sampling — discrete in **time**

You measure at instants, not continuously. Between samples, the signal is unknown.

Nyquist: sample at more than **twice** the highest frequency present, or the reconstruction is wrong.

Quantization — discrete in **value**

Each sample becomes one of a finite set of numbers. 12 bits gives $2^{12} = 4096$ levels, **0 to 4095**.

The error is up to half a step — inherent, not a defect.

Aliasing is not recoverable

Sample a 600 Hz tone at 1 kHz and it reappears as 400 Hz — and no amount of later processing can tell it from a real 400 Hz signal. The fix is an **anti-alias filter in hardware**, before the ADC.

$$V_{\text{in}} = \frac{\text{count}}{4095} \times V_{\text{ref}}$$

Count	Voltage ($V_{\text{ref}} = 3.3 \text{ V}$)	Meaning
0	0.00 V	Input at ground
1	0.81 mV	One step — the smallest detectable change
2048	1.65 V	Half scale
4095	3.30 V	Full scale

Resolution is a voltage, not a bit count

“12-bit” sounds abstract. What it *means* is that the smallest change you can see is $3.3/4095 \approx 0.8 \text{ mV}$. Anything finer is invisible to your program.

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

4 Design Challenge

```
// 1. Clocks
RCC->AHB1ENR |= (1U << 0);    // GPIOA
RCC->APB2ENR |= (1U << 8);    // ADC1

// 2. PA0 as analog -- MODER bits 11, the fourth mode from Chapter 8
GPIOA->MODER |= (3U << 0);

// 3. Sampling time: 480 cycles on channel 0
ADC1->SMPR2 |= (7U << 0);

// 4. Sequence: one conversion, channel 0
ADC1->SQR1 = 0;                // L = 0 means 1 conversion
ADC1->SQR3 = 0;                // first in sequence is channel 0

// 5. Turn it on
ADC1->CR2 |= (1U << 0);       // ADON
```

On this course PA0 is ADC1 channel 0 — and note MODER = 11, the analog mode that has been waiting since Chapter 8.

```
uint16_t ADC_Read(void)
{
    ADC1->CR2 |= (1U << 30);    // SWSTART
    while (!(ADC1->SR & (1U << 1)))
        ;                       // wait for EOC
    return ADC1->DR;             // reading DR
                                // clears EOC
}
```

1. **Start** — set SWSTART
2. **Wait** — poll EOC
3. **Read** — DR holds the result, and reading it clears the flag

This blocks, and Chapter 9 said why that is a shame

The processor does nothing during the conversion. A real design uses the **end-of-conversion interrupt**, or lets **DMA** move the result with no CPU involvement at all.

- The ADC charges an internal capacitor from your signal **before** converting.
- **Sampling time** is how long it is given to charge — 3 to 480 cycles.

```
ADC1->SMPR2 |= (7U « 0); // 480 cycles
```

The symptom is characteristic: readings **consistently low**, or changing when you touch the wiring. The fix is a longer sampling time — or a buffer amplifier.

Too short, and you read nonsense

A **high-impedance** source — a potentiometer, a sensor behind a large resistor — charges that capacitor slowly. Convert too early and you measure a *partly charged* capacitor, not your signal.

A thermocouple amplifier

Outputs 0–3.3 V for 0–400 °C.

```
float volts = (raw * 3.3f) / 4095.0f;  
float degC  = volts * (400.0f / 3.3f);
```

Or in one step:

```
float degC = raw * (400.0f / 4095.0f);
```

Note the f suffixes: 3.3f is a float; plain 3.3 is a double, which the Cortex-M4's FPU **cannot** do in hardware.

Can you actually resolve 1 °C?

400 °C over 4096 steps is **0.098 °C per step** — ten times better than needed.

So the ADC is not your limit. **Noise** is — and that is an analog design problem, not a programming one.

- An ADC approximates twice: **sampling** in time, **quantization** in value.
- **Nyquist**: sample above twice the highest frequency, or aliasing corrupts the result **irrecoverably**.
- 12-bit means **0–4095**; one step is $3.3/4095 \approx 0.8 \text{ mV}$. $V_{\text{in}} = (\text{count}/4095) \times V_{\text{ref}}$.
- Configure: **clocks**, MODER = 11 (**analog**), **sampling time**, **sequence**, then ADON.
- Read: SWSTART → wait EOC → read DR. Prefer the **interrupt** in real designs.
- **Sampling time** must suit the source impedance, or readings come out low.

Next: Chapter 12 – UART Communication

You can read the world. Next: telling someone about it.

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

4 Design Challenge

The brief

A thermocouple amplifier outputs 0–3.3 V for a range of 0–400 °C. The temperature must be reported to ± 1 °C, updated once per second, in an electrically noisy factory.

Design the acquisition chain: required ADC resolution, sampling rate, sampling time given a 10 k Ω source impedance, and whether averaging is needed – and if so, how many samples. Justify each number. Then state what limits the accuracy you can actually claim, and explain why a 12-bit converter does not by itself guarantee 12 bits of trustworthy measurement.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 11